

Winning Space Race with Data Science

Avery Jan

October 19, 2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Research: Falcon 9 first-stage landing success and operational factors.
- Workflow: SpaceX API + public launch records → cleaning → EDA/SQL → maps/dashboard → classification.
- Findings: analysis and model outputs require an executed notebook; do not report unverified accuracy.

Introduction

- Business problem: understand which flight characteristics affect the likelihood of a Falcon 9 first-stage landing.
- Questions: How do launch site, flight number, orbit, payload and launch year relate to success? Can models predict the landing outcome?
- Target: Class (1 = successful landing; 0 = unsuccessful landing).

Section 1

Methodology



Methodology

- 1. Collect historical SpaceX launch records using REST API and web scraping.
- 2. Standardize fields, remove duplicates, validate labels, and handle missing values.
- 3. Explore launch outcomes with plots and SQL queries.
- 4. Visualize launch sites with Folium; build interactive Dash views.
- 5. Fit, tune and compare classification models using a held-out test set.

Data Collection

- Data sources: SpaceX public REST API, publicly available launch listings and the course historical CSV.
- Collection pattern: API/HTML → tabular records → validation → standardized dataset → analysis.
- Required: supply actual outputs from successful API requests and scraped records.

Data Collection – SpaceX API

- GET SpaceX /v4/launches → HTTP validation → normalize JSON → check columns and row count.
- Notebook uses requests.get(timeout=30), raise_for_status() and pd.json_normalize().
- GitHub project: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

- INPUT → REQUEST → VALIDATE
- PARSE → NORMALIZE → DATASET

Data Collection - Scraping

- Retrieve Wikipedia Falcon 9 / Heavy launch listing → parse tables with BeautifulSoup → inspect rows → reconcile with API data.
- The scraper requires a network connection and may require adjustment if page structure changes.
- GitHub project: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

- INPUT → REQUEST → VALIDATE
- PARSE → NORMALIZE → DATASET

Data Wrangling

- Read dataset_part_2.csv → assert required columns → parse Date → coerce PayloadMass numeric → clean and inspect Class.
- Do not silently discard missing observations; document exclusions and class balance.
- GitHub project: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

EDA with Data Visualization

- Charts: landing success by site (bar); flight number by outcome/site (scatter); payload against orbit/site (scatter); yearly landing trend (line).
- Purpose: compare categories, identify changing success rates and detect potential trends.
- GitHub: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

EDA with SQL

- SQL performed in an in-memory SQLite table: group landing counts and rates by site/orbit, examine yearly trends, and query launch characteristics.
- Queries in this deck are templates until run on the source CSV.
- GitHub: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

Build an Interactive Map with Folium

- Create a Folium map with site coordinates; marker clusters aggregate nearby markers and popup text shows launches and rates.
- Use geospatial context to compare launch sites, not to infer causality.
- GitHub: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

Build a Dashboard with Plotly Dash

- Plotly charts: site-level landing rates and group size; interactive choices should include launch-site filters, payload ranges and success views.
- Dash application source is generated by the notebook but must be launched and verified separately.
- GitHub: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

Predictive Analysis (Classification)

- Predict Class using payload, flight number, orbit and launch site.
- Preprocess with numeric scaling and one-hot encoding; compare classifier pipelines with held-out evaluation.
- Use cross-validation for tuning and retain test results to avoid overstatement.
- GitHub: https://github.com/AveryJan/SpaceX-Data-Science-Capstone

Results

- EDA: generate figures using actual source data.
- Interactive analytics: capture Folium and Plotly Dash screenshots only after execution.
- Prediction: report test accuracy, confusion matrix and per-class precision/recall after a successful run.
- Verification: notebook currently has no stored outputs.

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

- FlightNumber (x) vs LaunchSite (y), colored by landing Class.
- Interpretation: helps inspect experience-related patterns within each site.
- Chart pending successful dataset load and rendering.

Payload vs. Launch Site

- PayloadMass (x) vs LaunchSite (y), colored by landing outcome.
- Interpretation: compares payload distributions across sites.
- Chart pending successful dataset load and rendering.

Success Rate vs. Orbit Type

- Landing success rate = AVG(Class) grouped by Orbit, plotted as a bar chart.
- Different mission profiles may correspond to different outcomes.
- Verified rates pending notebook run.

Flight Number vs. Orbit Type

- Scatter FlightNumber against orbit categories, colored by Class.
- Reveals when orbit categories enter the launch history.
- Observed points pending dataset execution.

Payload vs. Orbit Type

- Scatter PayloadMass against orbit categories, colored by Class.
- Compares payload distribution and landing results by orbit.
- Observed points pending dataset execution.

Launch Success Yearly Trend

- Line chart: yearly success percentage = 100 × AVG(Class) grouped by launch year.
- Use Date-derived year and label record coverage.
- Computed trend pending execution.

All Launch Site Names

- SQL: SELECT DISTINCT LaunchSite FROM launches ORDER BY LaunchSite;
- Returns unique launch-site names in the loaded dataset.
- Actual result pending SQLite query execution.

Launch Site Names Begin with 'CCA'

- SQL: SELECT * FROM launches WHERE LaunchSite LIKE 'CCA%' LIMIT 5;
- Filters Cape Canaveral records by prefix.
- Actual five rows pending query execution.

Total Payload Mass

- SQL: SELECT SUM(PayloadMass) AS total_payload_kg FROM launches WHERE Customer LIKE '%NASA%';
- Requires a source table with a verified customer field.
- Result not reported until source schema is confirmed.

Average Payload Mass by F9 v1.1

- SQL: SELECT AVG(PayloadMass) AS avg_payload_kg FROM launches WHERE BoosterVersion LIKE 'F9 v1.1%';
- Requires a verified booster-version column.
- Result pending schema verification and query execution.

First Successful Ground Landing Date

- SQL: SELECT MIN(Date) FROM launches WHERE LandingOutcome = 'Success (ground pad)';
- Find the earliest successful ground-pad landing in a verified outcome table.
- Result pending source-field verification.

Successful Drone Ship Landing with Payload between 4000 and 6000

- SQL: SELECT BoosterVersion FROM launches WHERE LandingOutcome='Success (drone ship)' AND PayloadMass BETWEEN 4000 AND 6000;
- Filter correct units and outcomes before interpretation.
- Result pending dataset execution.

Total Number of Successful and Failure Mission Outcomes

- SQL: SELECT Class, COUNT(*) AS total FROM launches GROUP BY Class;
- Counts outcome-label distribution.
- Actual counts pending execution.

Boosters Carried Maximum Payload

- SQL: SELECT BoosterVersion, PayloadMass FROM launches ORDER BY PayloadMass DESC LIMIT 1;
- Identifies the maximum-payload launch in a source with booster labels.
- Result pending source verification.

2015 Launch Records

- SQL: SELECT * FROM launches WHERE substr(Date,1,4)='2015' AND LandingOutcome LIKE 'Failure%';
- Inspect booster type, launch site and 2015 landing failures.
- Result pending execution.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- SQL: SELECT LandingOutcome, COUNT(*) AS n FROM launches WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY LandingOutcome ORDER BY n DESC;
- Rank outcomes within period after field validation.


Section 3
Launch Sites
Proximities Analysis

Folium — Launch Site Markers

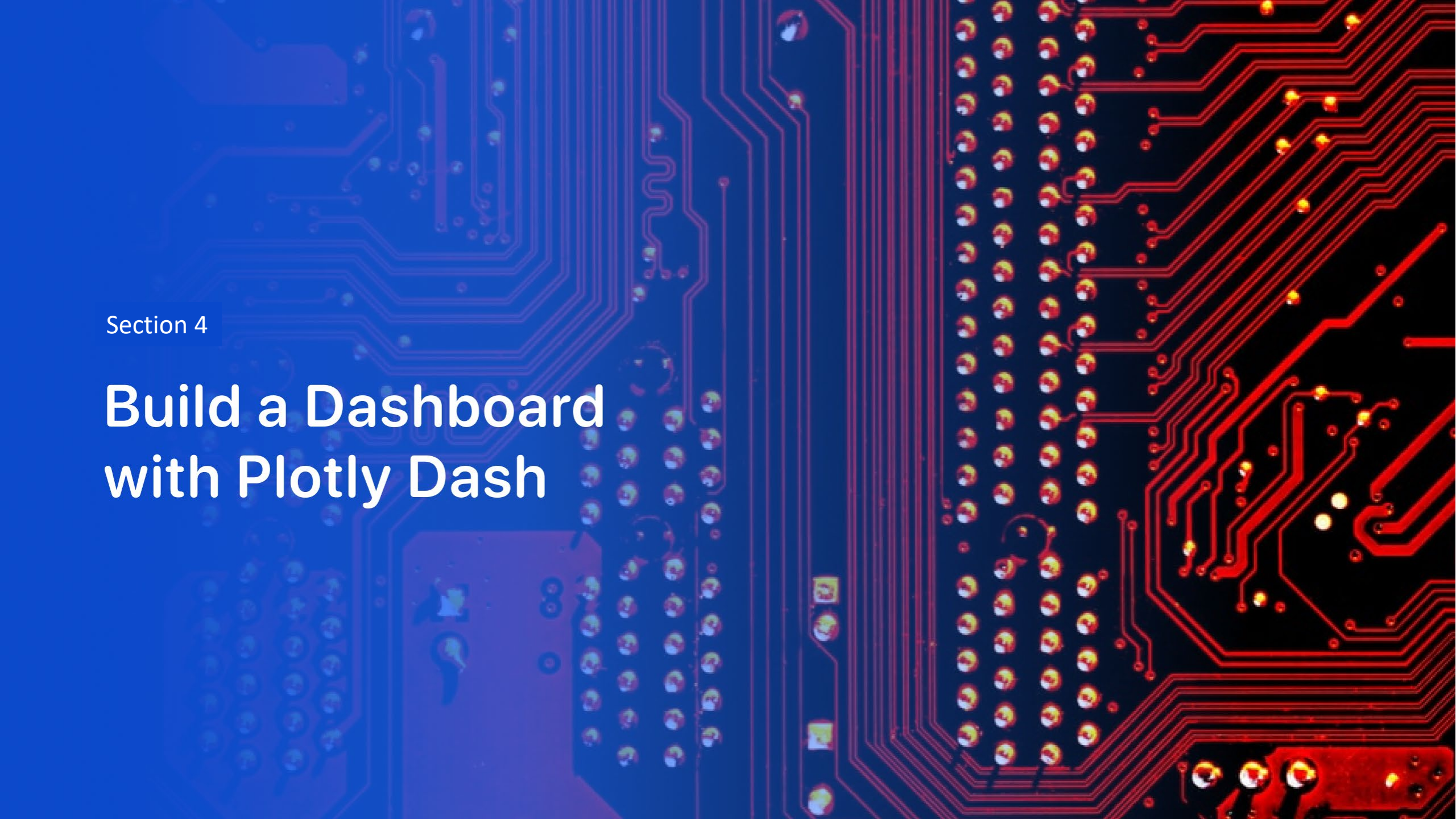
- Launch-site location markers
- Map source: latitude and longitude aggregated by LaunchSite.
- MarkerCluster and popup text display launch counts and success rates.
- Actual map screenshot pending notebook execution.

Folium – Landing Rate Map

- Launch-site outcome comparison
- Use colored markers or circles to display historical landing rates.
- Include legend and explanatory site labels.
- Actual map screenshot pending execution.

Folium – Geographic Insights

- Launch-site geographic interpretation
- Study proximity of operating locations using visible sites and coastlines.
- Do not infer launch trajectory from marker positions alone.
- Map screenshot pending execution.


Section 4
Build a Dashboard with Plotly Dash

Dash — Site-Level Results

- Dash launch outcome dashboard
- Bar chart: landing rate by site; interactive selection filters selected launch site.
- Screenshots require running the Dash server.
- Screenshot pending execution.

Dash — Outcome Pie Chart

- Dash landing outcomes by launch site
- Pie chart: success versus failure for user-selected launch site.
- Show denominator to avoid misleading comparisons.
- Screenshot pending execution.

Dash — Payload Interactions

- Dash payload vs launch outcome
- Scatter chart uses PayloadMass and Class/BoosterVersion with a payload-range slider.
- The app must be run and tested in a browser.
- Screenshot pending execution.

Section 5

Predictive Analysis (Classification)



Classification Accuracy

- Compare held-out classification performance across trained models.
- Display measured accuracy, precision, recall and/or F1; do not invent model scores.
- No persisted model outputs were available in the recovered notebook.

Confusion Matrix

- Confusion matrix: actual Class by predicted Class on held-out test records.
- Interpret true positives, true negatives, false positives and false negatives.
- A verified matrix requires running model evaluation.

Conclusions

- • OneLake-style centralization is not part of this project: analysis is on Falcon 9 data.
- • Launch site, flight experience, payload and orbit are candidates for prediction.
- • Data-quality checks and leakage-aware validation matter for credible evaluation.
- • Actual model ranking and numeric conclusions remain unverified until execution.

Appendix

- Source project: https://github.com/AveryJan/SpaceX-Data-Science-Capstone
- Notebook: Falcon9_Capstone_Notebook_Jan.ipynb (12 Python code cells).
- Report: Data_Science_Capstone_Project_Report_Jan.pdf.
- Execution prerequisites: dataset_part_2.csv, Python libraries, network access for API/scraping.
- Full code is reproduced on subsequent appendix slides.

Thank you!



Appendix — Notebook code cell 2 (1/2)

```
# Run once if your environment lacks any dependencies:
# %pip install -q pandas numpy requests beautifulsoup4 lxml matplotlib seaborn scikit-learn folium plotly dash
import io, sqlite3, warnings
import numpy as np, pandas as pd, requests
import matplotlib.pyplot as plt
import seaborn as sns
from bs4 import BeautifulSoup
from IPython.display import display, HTML
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
sns.set_theme(style='whitegrid')
SEED = 42
BASE = 'https://raw.githubusercontent.com/elgels/IBM-Data-Science-Capstone/main/data/'
def load_csv(filename):
```

Appendix — Notebook code cell 2 (2/2)

```
try:
    return pd.read_csv(filename)
except FileNotFoundError:
    try:
        res=requests.get(BASE + filename, timeout=35); res.raise_for_status()
        return pd.read_csv(io.BytesIO(res.content))
    except Exception as exc:
        raise RuntimeError(f'Download {filename} or place it next to this notebook, then rerun.') from exc
```

Appendix — Notebook code cell 4 (1/1)

```
api_url='https://api.spacexdata.com/v4/launches'
try:
    response=requests.get(api_url,timeout=30); response.raise_for_status()
    launches=pd.json_normalize(response.json())
    print('SpaceX API response:',response.status_code,'launch records:',len(launches))
    display(launches[['name','date_utc','success']].head() if {'name','date_utc','success'}.issubset(launches.colum
ns) else launches.head())
except requests.RequestException as exc:
    print('API temporarily unavailable; historical analysis below is independent:',exc)
    launches=pd.DataFrame()
```

Appendix — Notebook code cell 6 (1/1)

```
wiki_url='https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches'
try:
    page=requests.get(wiki_url,headers={'User-Agent':'EducationalDataScienceProject/1.0'},timeout=35)
    page.raise_for_status()
    soup=BeautifulSoup(page.text,'html.parser')
    tables=soup.select('table.wikitable')
    print('HTML tables with class wikitable:',len(tables))
    print('Example table headings:',[x.get_text(' ',strip=True) for x in tables[0].select('th')[:8]] if tables else
 [])
    extracted=[]
    for table in tables:
        for tr in table.select('tr'):
            values=[col.get_text(' ',strip=True) for col in tr.select('th, td')]
            if len(values)>=4: extracted.append(values[:6])
    print('Structured rows extracted:',len(extracted))
    display(pd.DataFrame(extracted[:10]))
except requests.RequestException as exc: print('Scraping unavailable:',exc)
```

Appendix — Notebook code cell 8 (1/1)

```
raw=load_csv('dataset_part_2.csv')
assert {'Class','FlightNumber','LaunchSite','Orbit','PayloadMass'}.issubset(raw.columns)
df=raw.copy()
df['Date']=pd.to_datetime(df['Date'],errors='coerce')
df['PayloadMass']=pd.to_numeric(df['PayloadMass'],errors='coerce')
df['Class']=pd.to_numeric(df['Class'],errors='coerce')
df=df[df['Class'].isin([0,1])].copy()
df['Class']=df['Class'].astype(int)
print('Shape:',df.shape,'duplicates:',df.duplicated().sum())
print('Class frequency:'); display(df['Class'].value_counts().rename(index={0:'not successful',1:'successful'}))
print('Missing values:'); display(df.isna().sum().sort_values(ascending=False).head(12))
print('Date span:',df['Date'].min(),df['Date'].max())
assert df['Class'].nunique()==2
```

Appendix — Notebook code cell 10 (1/1)

```
with sqlite3.connect(':memory:') as con:
    df.to_sql('launches',con,index=False,if_exists='replace')
    for label,q in {
      'launch site performance': "SELECT LaunchSite, COUNT(*) AS launches, ROUND(AVG(Class)*100,1) AS landing_pct F
ROM launches GROUP BY LaunchSite ORDER BY launches DESC",
      'orbit performance': "SELECT Orbit, COUNT(*) AS launches, ROUND(AVG(Class)*100,1) AS landing_pct FROM launche
s GROUP BY Orbit ORDER BY launches DESC",
      'booster reuse': "SELECT Reused, COUNT(*) AS launches, ROUND(AVG(Class)*100,1) AS landing_pct FROM launches G
ROUP BY Reused",
      'annual trend': "SELECT SUBSTR(Date,1,4) AS year,COUNT(*) AS launches,ROUND(AVG(Class)*100,1) AS landing_pct
FROM launches GROUP BY SUBSTR(Date,1,4) ORDER BY year"
    }.items():
        print('\n'+label); display(pd.read_sql_query(q,con))
```

Appendix — Notebook code cell 12 (1/1)

```
fig,axs=plt.subplots(2,2,figsize=(13,9))
sns.barplot(data=df.groupby('LaunchSite',as_index=False)['Class'].mean(),x='LaunchSite',y='Class',ax=axs[0,0])
axs[0,0].set(title='Historical landing rate by site',ylabel='Rate',xlabel='Launch site');axs[0,0].tick_params(axis=
'x',rotation=20)
sns.barplot(data=df.groupby('Orbit',as_index=False)['Class'].mean().sort_values('Class',ascending=False),x='Orbit',
y='Class',ax=axs[0,1]);axs[0,1].tick_params(axis='x',rotation=40);axs[0,1].set(title='Historical landing rate by or
bit',ylabel='Rate')
sns.scatterplot(data=df,x='FlightNumber',y='PayloadMass',hue='Class',ax=axs[1,0]);axs[1,0].set(title='Payload vs. f
light number',ylabel='Payload (kg)')
annual=df.dropna(subset=['Date']).assign(Year=lambda d:d.Date.dt.year).groupby('Year')['Class'].mean()
annual.plot(marker='o',ax=axs[1,1]);axs[1,1].set(title='Historical landing rate by year',ylabel='Rate',ylim=(-.05,1
.05))
plt.tight_layout();plt.show()
```

Appendix — Notebook code cell 14 (1/1)

```
import folium
from folium.plugins import MarkerCluster
sites=df.dropna(subset=['Latitude','Longitude']).groupby(['LaunchSite','Latitude','Longitude'],as_index=False).agg(
launches=('Class','size'),success_rate=('Class','mean'))
m=folium.Map(location=[31.5,-99],zoom_start=4,tiles='CartoDB positron')
for row in sites.itertuples():
    folium.CircleMarker(location=[row.Latitude,row.Longitude],radius=8+min(row.launches,50)/5,
      popup=f'{row.LaunchSite}: {row.launches} launches, {row.success_rate:.1%} landing rate',
      color='#167d8d',fill=True,fill_opacity=.75).add_to(m)
display(m)
m.save('Falcon9_Launch_Sites_Map.html')
```

Appendix — Notebook code cell 16 (1/1)

```
import plotly.express as px
site_stats=df.groupby('LaunchSite')['Class'].agg(['mean','size']).reset_index()
fig=px.bar(site_stats,x='LaunchSite',y='mean',color='size',title='Landing success rate by launch site',labels={'mea
n':'Success rate','size':'Launches'})
fig.show()
fig2=px.scatter(df,x='PayloadMass',y='FlightNumber',color=df['Class'].map({0:'No landing',1:'Landing'}),hover_data=
['LaunchSite','Orbit'],title='Payload and flight number vs. recorded landing result')
fig2.show()
```

Appendix — Notebook code cell 17 (1/2)

```
# Save a working Dash app as a separate Python source file.
dash_source = r"""import pandas as pd
import plotly.express as px
from dash import Dash, dcc, html, Input, Output
DATA_URL = "https://raw.githubusercontent.com/elgels/IBM-Data-Science-Capstone/main/data/dataset_part_2.csv"
data = pd.read_csv(DATA_URL)
app = Dash(__name__)
app.layout = html.Div([
    html.H2("Falcon 9 historical landing dashboard"),
    dcc.Dropdown(id="site", options=[{"label":"All Sites","value":"ALL"}] +
                 [{"label":s,"value":s} for s in sorted(data.LaunchSite.unique())], value="ALL"),
    dcc.RangeSlider(id="payload",min=0,max=16000,step=500,value=[0,16000],
                    marks={0:"0kg",8000:"8,000kg",16000:"16,000kg"}),
    dcc.Graph(id="success"), dcc.Graph(id="payloadgraph")
])
@app.callback(Output("success","figure"),Output("payloadgraph","figure"),
              Input("site","value"),Input("payload","value"))
def update(site,payload):
    s=data if site=="ALL" else data[data.LaunchSite==site]
    s=s[s.PayloadMass.between(payload[0],payload[1])]
    if s.empty: return px.scatter(title="No matching launches"),px.scatter(title="No matching launches")
    group=s.groupby("LaunchSite",as_index=False).Class.mean()
```

Appendix — Notebook code cell 17 (2/2)

```
    return (px.bar(group,x="LaunchSite",y="Class",title="Landing success by site"),
            px.scatter(s,x="PayloadMass",y="FlightNumber",color="Class",title="Payload and outcome"))
if __name__ == "__main__": app.run(debug=True)
"""
with open('Falcon9_Dash_App.py','w',encoding='utf-8') as f: f.write(dash_source)
print('Saved Falcon9_Dash_App.py; start with: python Falcon9_Dash_App.py')
```

Appendix — Notebook code cell 19 (1/2)

```
from sklearn.preprocessing import OneHotEncoder
feature_cols=['PayloadMass','FlightNumber','Orbit','LaunchSite']
X=df[feature_cols].copy(); y=df['Class'].copy()
num=['PayloadMass','FlightNumber']; cat=['Orbit','LaunchSite']
transformer=ColumnTransformer([
 ('num',Pipeline([('fill',SimpleImputer(strategy='median')),('scale',StandardScaler())]),num),
 ('cat',Pipeline([('fill',SimpleImputer(strategy='most_frequent')),('onehot',OneHotEncoder(handle_unknown='ignore')
)]),cat)
])
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=.25,random_state=SEED,stratify=y)
models={
  'Logistic regression':(LogisticRegression(max_iter=2000),{'classifier__C':[.01,.1,1,10]}),
  'Support vector machine':(SVC(),{'classifier__kernel':['linear','rbf'],'classifier__C':[.1,1,10]}),
  'Decision tree':(DecisionTreeClassifier(random_state=SEED),{'classifier__max_depth':[2,3,5,None],'classifier__min
_samples_leaf':[1,3,5]}),
  'K nearest neighbors':(KNeighborsClassifier(),{'classifier__n_neighbors':[3,5,7,9]})
}
cv=StratifiedKFold(n_splits=5,shuffle=True,random_state=SEED)
results=[]; fitted={}
for name,(estimator,params) in models.items():
    pipe=Pipeline([('features',transformer),('classifier',estimator)])
    search=GridSearchCV(pipe,params,scoring='accuracy',cv=cv,n_jobs=-1)
```

Appendix — Notebook code cell 19 (2/2)

```
    search.fit(X_train,y_train)
    pred=search.predict(X_test)
    results.append({'model':name,'cv_accuracy':search.best_score_,'test_accuracy':accuracy_score(y_test,pred),'para
meters':str(search.best_params_)})
    fitted[name]=search.best_estimator_
results_df=pd.DataFrame(results).sort_values('test_accuracy',ascending=False)
display(results_df)
print('Class majority baseline:',max(y_test.mean(),1-y_test.mean()))
best_name=results_df.iloc[0]['model'];best=fitted[best_name]
print('Best model by held-out test accuracy (descriptive only):',best_name)
print(classification_report(y_test,best.predict(X_test),zero_division=0))
ConfusionMatrixDisplay.from_estimator(best,X_test,y_test,display_labels=['No landing','Landing'],cmap='Blues');plt.
title(best_name);plt.show()
```

Appendix — Notebook code cell 21 (1/1)

```
chron=df.dropna(subset=['Date']).sort_values('Date').reset_index(drop=True)
cut=int(len(chron)*.75)
tr,te=chron.iloc[:cut],chron.iloc[cut:]
if tr['Class'].nunique()>1 and len(te)>0:
    temporal=Pipeline([('features',transformer),('classifier',LogisticRegression(max_iter=2000))])
    temporal.fit(tr[feature_cols],tr['Class'])
    print('Earlier period:',tr.Date.min().date(),'-',tr.Date.max().date())
    print('Later period:',te.Date.min().date(),'-',te.Date.max().date())
    print('Temporal test accuracy:',round(accuracy_score(te['Class'],temporal.predict(te[feature_cols])),3))
else: print('Temporal split insufficient for classification')
```

Appendix — Notebook code cell 24 (1/4)

```
from matplotlib.backends.backend_pdf import PdfPages
from matplotlib.figure import Figure
from pathlib import Path
from textwrap import wrap
export_dir=Path('falcon9_submission_evidence');export_dir.mkdir(exist_ok=True)
chart_paths=[]
def evidence_plot(name, plot_func):
    fig,ax=plt.subplots(figsize=(10,5.7)); plot_func(ax)
    fig.tight_layout(); path=export_dir/(name+'.png'); fig.savefig(path,dpi=165,bbox_inches='tight');plt.close(fig)
    chart_paths.append(path)
evidence_plot('01_success_by_site',lambda ax: df.groupby('LaunchSite')['Class'].mean().sort_values().plot.barh(ax=a
x,title='Landing success rate by launch site',xlabel='Success rate'))
evidence_plot('02_success_by_orbit',lambda ax: df.groupby('Orbit')['Class'].mean().sort_values().plot.barh(ax=ax,ti
tle='Landing success rate by orbit',xlabel='Success rate'))
evidence_plot('03_payload_scatter',lambda ax: sns.scatterplot(data=df,x='PayloadMass',y='FlightNumber',hue='Class',
ax=ax))
evidence_plot('04_yearly_trend',lambda ax: df.assign(Year=df.Date.dt.year).groupby('Year')['Class'].mean().plot(ax=
ax,marker='o',title='Historical landing rate by year',ylabel='Success rate'))
evidence_plot('05_model_comparison',lambda ax: results_df.set_index('model')[['cv_accuracy','test_accuracy']].plot.
barh(ax=ax,title='Model comparison (accuracy)',xlim=(0,1)))
evidence_plot('06_confusion_matrix',lambda ax: ConfusionMatrixDisplay.from_estimator(best,X_test,y_test,display_lab
els=['No','Yes'],ax=ax,cmap='Blues',colorbar=False))
```

Appendix — Notebook code cell 24 (2/4)

```
from reportlab.pdfgen import canvas as pdfcanvas
from reportlab.lib.utils import ImageReader
from reportlab.lib.pagesizes import landscape,letter
from reportlab.lib.colors import HexColor
output_pdf=export_dir/'Falcon9_Executed_Results_Submission.pdf'
c=pdfcanvas.Canvas(str(output_pdf),pagesize=landscape(letter));W,H=landscape(letter)
def sheet(title,lines):
    c.setFillColor(HexColor('#142947'));c.rect(0,H-80,W,80,stroke=0,fill=1)
    c.setFillColorRGB(1,1,1);c.setFont('Helvetica-Bold',22);c.drawString(42,H-52,title[:65]); c.setFillColorRGB(.1,
.15,.23)
    text=c.beginText(45,H-108);text.setFont('Helvetica',11);text.setLeading(16)
    for line in lines:
        for chunk in wrap(str(line),104):text.textLine(chunk)
        text.textLine('')
    c.drawText(text);c.showPage()
sheet('Falcon 9 Landing Prediction | Executive summary',[
 'Business question: estimate historical first-stage landing outcome using characteristics known before launch.',
 'Source: IBM Skills Network historical teaching data (community-hosted course-compatible mirror).',
 f'Sample size: {len(df)}; success rate: {df.Class.mean():.1%}; date range: {df.Date.min().date()} to {df.Date.max(
).date()}.',
 f'Best by held-out accuracy: {best_name}; accuracy: {accuracy_score(y_test,best.predict(X_test)):.3f}.',
 'Caution: the best model was selected using the test split; compare models by cross-validation for unbiased select
ion.' ])
```

Appendix — Notebook code cell 24 (3/4)

```
sheet('Data collection, wrangling and SQL',[
 'REST API source: api.spacexdata.com/v4/launches (independent live inspection).',
 'Web scraping source: en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches.',
 f'Historical CSV shape: {df.shape}; duplicated rows: {df.duplicated().sum()}.',
 'Missing fields: '+str(df.isna().sum().sort_values(ascending=False).head(7).to_dict()),
 'SQL: sqlite3 queries grouped by launch site, orbit, booster reuse and year; corresponding aggregations plotted on
 subsequent pages.' ])
for path in chart_paths:
    c.setFillColor(HexColor('#142947'));c.rect(0,H-64,W,64,stroke=0,fill=1)
    c.setFillColorRGB(1,1,1);c.setFont('Helvetica-Bold',19);c.drawString(37,H-42,path.stem.replace('_',' ').title()
)
    c.drawImage(ImageReader(str(path)),36,45,width=W-72,height=H-130,preserveAspectRatio=True,anchor='c')
    c.showPage()
for _,r in results_df.iterrows():
    sheet(str(r['model'])+' | evaluation',[f"Cross-validated accuracy: {r['cv_accuracy']:.3f}",f"Hold-out accuracy:
 {r['test_accuracy']:.3f}",f"Hyperparameters: {r['parameters']}", 'Evaluation: 75/25 stratified holdout with 5-fold
 tuning on the training portion.'])
sheet('Conclusion and responsible interpretation',[
 'Landing outcomes vary in the historical sample across site, orbit and launch year (see charts).',
 'The sample is historical, not a current operational data feed; the class label includes course-defined ocean outc
omes.',
 'Limited sample size, chronological drift, confounding, and uncertainty about future mission conditions limit gene
ralization.',
```

Appendix — Notebook code cell 24 (4/4)

```
 'Recommendations: validate on future launches, recalibrate model probabilities, monitor drift and measure false-po
sitive risk.',
 'Full executable project is in the companion .ipynb notebook; interactive Folium map is saved as HTML, and Dash ap
p as Python.' ])
c.save();print('SUBMIT THIS EXECUTED PDF:',output_pdf.resolve())
print('Evidence plots:',len(chart_paths))
```